LAB 3: Design and Implementation

1. OBJECTIVES

The following objectives are to be started during Lab #3 and completed by the beginning of Lab #4 unless otherwise stated.

1.1 Transform the analysis model into a design model
1.2 Determine proper system architecture
1.3 Address key design issues and use appropriate design patterns
1.4 AI-based project initialisation
1.5 Start implementing your design model

2. INTRODUCTION

2.1 During software design, the analysis model is transformed into the corresponding design model. In object-oriented methodology, the important classes in the analysis model can be mapped onto the design model.

2.2 The boundary between analysis and design stages in the SDLC is fuzzy. At the tail end of analysis, you were already beginning to think about some of the design issues listed below. You will continue to refine the dialog map from Lab #2.

2.3 Make design decisions to incorporate good principles such as encapsulation, loose coupling and high cohesion. Good design decisions at this stage will improve reusability, extensibility and maintainability.

2.4 Once your design model becomes stable, you can start mapping your design model onto code. That is, start implementing your system.

If you implement your design using Java or other Object-Oriented languages

- each UML class is mapped onto a Java class
- the generalization relationship is mapped to an extends keyword
- the interface realization relationship is mapped onto an implements keyword
- an x-to-many association is mapped onto a Collection.

2.5 As you start implementation, keep in mind the traceability from requirements to design, then to implementation and test cases.

2.5 In addition to software tools provided by the SWLAB, you can choose any technologies that you deem relevant to the design and implementation of your applications. You need to justify your choice in the project demo and final documentations. Remember that software tools not provided from the School come with no School support so be sure that they work for you before committing to them.

3. PROCEDURE

3.1 Transform analysis model into design model

3.1.1 Modify your conceptual model from Lab #2 to make a draft design class model.

3.1.2 Determine appropriate system architecture

3.1.3 Follow the heuristics from page 341 ~ 345 of Fox:

- Add a start-up class (which performs system initialization)
- Add control classes and distribute the logic and coordination responsibilities amongst them
- Add container classes to hold collection of objects, e.g. List
- Apply design patterns where appropriate

3.1.4 Re-visit your dialog map from Lab #2. Add detail e.g. attributes and operations to your class model.

3.2 Address key design issues

3.2.1 In your design, address key design issues listed in Chapter 7.4 of Bruegge:

- Identifying and storing persistent data
- Providing access control

3.2.2 Apply design patterns where appropriate.

3.2.3 Add detail to the class diagram of your design model.

3.3 AI-based project initialisation

3.3.1 It is common for development teams to revisit their planned technology stack as a project evolves its design and nears the implementation phase. Repeat the technology stack prompting exercise from Lab#2 (3.4.2) with your updated class diagram (and your Lab#1 system description). Try to use the same AI model and copied text prompt as the previous lab, in a fresh chat session, if possible.

If you have decided to change your preferred programming language and/or any major frameworks since the previous lab, you may instead prompt the AI with this new information.

3.3.2 How similar are the AI's recommendations to those of the previous exercise - are they exactly the same, mostly the same, or completely different?

(1) If there are any differences in the AI's output, and your prompt/technology preferences from the previous lab did not change, can you identify any relevant differences between your two input class diagrams that might have caused the AI's recommendations to change?

(2) If you have prompted the AI to give recommendations for a different programming language/framework in comparison to the previous lab, ask the AI a follow-up question - tell it the technology preferences you expressed in the previous lab and ask it to compare them to your current choices. Does the AI agree or disagree with your decision to change?

3.3.3 Using your class diagram as input, and for your preferred choice of programming language, try to get an AI to generate a minimal skeleton template for your project, consisting of a folder hierarchy containing files that are empty except for TODO comments. Carefully instruct the AI not to put any code inside the files.

This is a perfect opportunity to experiment with Cline, if you have installed VSCode or IntelliJ as suggested by the preparatory lab materials on AI use. You can also use a Web interface such as ChatGPT - as of the time of writing, the free Web chat tier of ChatGPT is capable of carrying out this task, although careful prompting is needed to get it to return the files in a downloadable format (such as .zip).

3.3.4 Critically evaluate the AI's output.

- To what extent do you agree with its choices?
- Has it missed something obvious or failed to produce anything reasonable?
- Did it add anything to the files beyond what was requested (such as initial implementation code)? If so, describe what.

3.3.5 Prepare a short PDF report containing
(1) for 3.3.1: your text prompts, the AI's responses, and a few sentences answering the questions posed in 3.3.2.
(2) for 3.3.3: your text prompts, the AI's textual responses, and answers to the questions posed in 3.3.4.

Also upload a .zip file containing the folders and files outputted by the AI as part of 3.3.3 - make sure this can be distinguished from your "full" application skeleton submission (see below).

3.4 Implement your design in code

3.4.1 From the class diagram, prepare the full skeleton code, with simple top-level declarations, important functions/objects, and import structure sketched out. You may extend the AI output from 3.3.3 if you wish, but make sure to carefully check it first. Once you have prepared a skeleton, you should begin to implement the application's behavior as documented in the Use Case model and Dialog Map. You will finish your implementation as part of Lab#4, but you should start now!

3.4.2 You may wish to organize the classes into packages or folders, following the stereotypes in the design class model.

3.4.3 Document the classes and the key public methods to convey design intent and usage using Javadoc or similar techniques for other programming languages.

3.4.4 To facilitate team collaboration, upload the source code and libraries into Github. Your teammates can then check out the code.

Note Github is an industry practice for collaborative software development. It is very different from document sharing services such as Dropbox. Similar services include gitlab, bitbucket, etc. You may use other repositories, but you should regularly upload your work products to your team's Github repository.

4. DELIVERABLES

- Complete Use Case model (diagram and descriptions)
- Design Model
    - Class diagram
    - Dialog map
- System architecture
- Application skeleton (after 3.4 hand-modifications). Submission of an in-progress prototype is also acceptable so long as the skeleton structure of the overall project is clear.
- Your PDF report as detailed in 3.3.5.

Note Previous versions of this lab also required the submission of "sequence diagrams". This is no longer required, and sequence diagrams are no longer examinable.

Please submit the deliverables to your Github repository (under the folder "lab3") before the start of Lab#4. Your Lab Supervisor will need to see and discuss these deliverables with you during Lab #4.

Note Lab#4 takes place 5 weeks after your Lab#3 (i.e. teaching week 10 or 11).
There is **no lab** on teaching week 8 or 9.

5. REFERENCES

- Introduction to Software Engineering Design: Processes, Principles and Patterns with UML2 – C. Fox
- Object-Oriented Software Engineering: Using UML, Patterns and Java – B. Bruegge and A. Dutoit